

Faceted Data

The paper presents a research-driven approach to preventing data leakage through dynamic information flow control, shifting the focus from access control to the propagation of information during execution. This is a substantial strength, as many real-world breaches occur after authorised access has already been granted and data begin to flow through complex code paths. Mechanisms such as Generalised Multiple Facets (GMF), Optimised GMF (OGMF), and Termination-Sensitive Multiple Facets (TSMF) aim to enforce confidentiality across security levels and mitigate covert channels, including those arising from program termination behaviour.

Conceptually, the model is rigorous and offers formal guarantees about information flow, including termination-insensitive non-interference and—under TSMF—termination-sensitive non-interference. It represents a more systematic and theoretically grounded approach than traditional access-control-centric mechanisms, and the support for arbitrary finite lattices enhances its flexibility across different policy models and application domains. Notably, the paper demonstrates that such guarantees can be implemented at library level within an existing programming language, rather than requiring a fully redesigned runtime, which increases its potential applicability. These properties align closely with modern security paradigms such as Zero Trust, where trust is not derived from network location but enforced continuously through data-centric controls.

However, in practical terms, the approach is difficult to justify outside high-assurance or research contexts. The reliance on multi-execution or execution branching introduces substantial computational overhead, even with the optimisations proposed in OGMF. Empirical work on dynamic information flow for JavaScript indicates that tracking implicit flows and maintaining faceted memories can incur significant runtime and memory costs. Moreover, the implementation complexity—particularly around faceted values, program counter tracking, and state merging—creates a high barrier to adoption and increases the risk of subtle implementation errors that could undermine the security guarantees. This reflects a fundamental trade-off between strong confidentiality guarantees and system performance and usability, a recurring challenge in secure system design. At the same time, the model offers clear advantages in assume breach scenarios, as it limits the ability of compromised components to exfiltrate sensitive data. In this sense, it also contributes to the broader objective of digital sovereignty by enabling stronger technical control over how and where sensitive information can be observed.

A simple example illustrates the enforcement mechanism. Consider the conditional *if* ($h > 0$) *then* $l := 1$ *else* $l := 0$, where h is confidential and l is public. Under a faceted semantics, the guard $h > 0$ and the assignment to l are evaluated in multiple security contexts. A low-security observer only ever sees a default or sanitised outcome projected from a faceted value, while a high-security context reveals the true branch result. This prevents the low-level observer from inferring any information about h through control flow.

In Python, I would implement a simplified prototype using a small security lattice, security labels on variables, faceted values, and a projection function, combined with a basic multi-execution engine or a faceted interpreter for a restricted subset of the language. This would not be suitable for production deployment, but it would capture the core principle of

faceted information flow in a way that is feasible for an educational setting and aligns with secure software development themes of rigorous design and defence-in-depth.

Victor Angelier